

Chapter 5. A2A (Agent-to-Agent) 통합

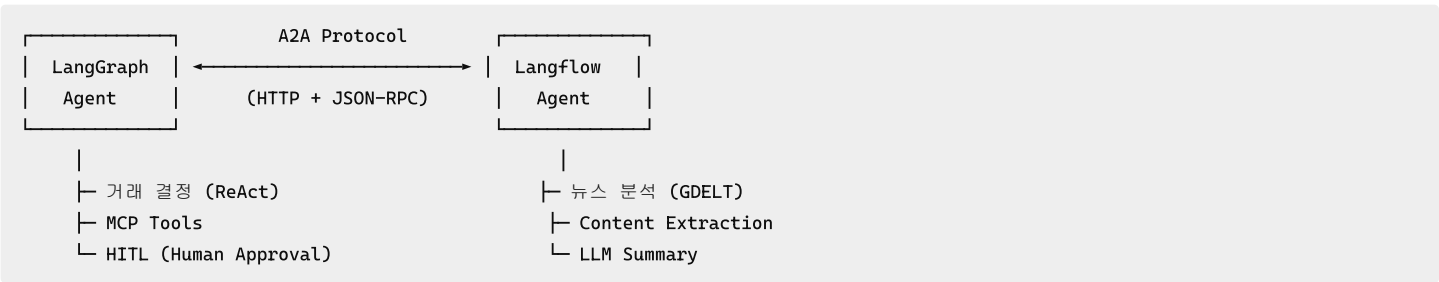
📖 학습 목표

- A2A 프로토콜의 개념과 필요성 이해
- LangGraph Agent와 Langflow Agent 간 통신 구현
- Bootstrap 패턴을 통한 Agent Discovery
- 실전 Trading Bot에서 뉴스 분석 통합

💡 A2A란?

Agent-to-Agent Protocol

여러 AI Agent가 서로 통신하고 협력할 수 있게 하는 표준 프로토콜

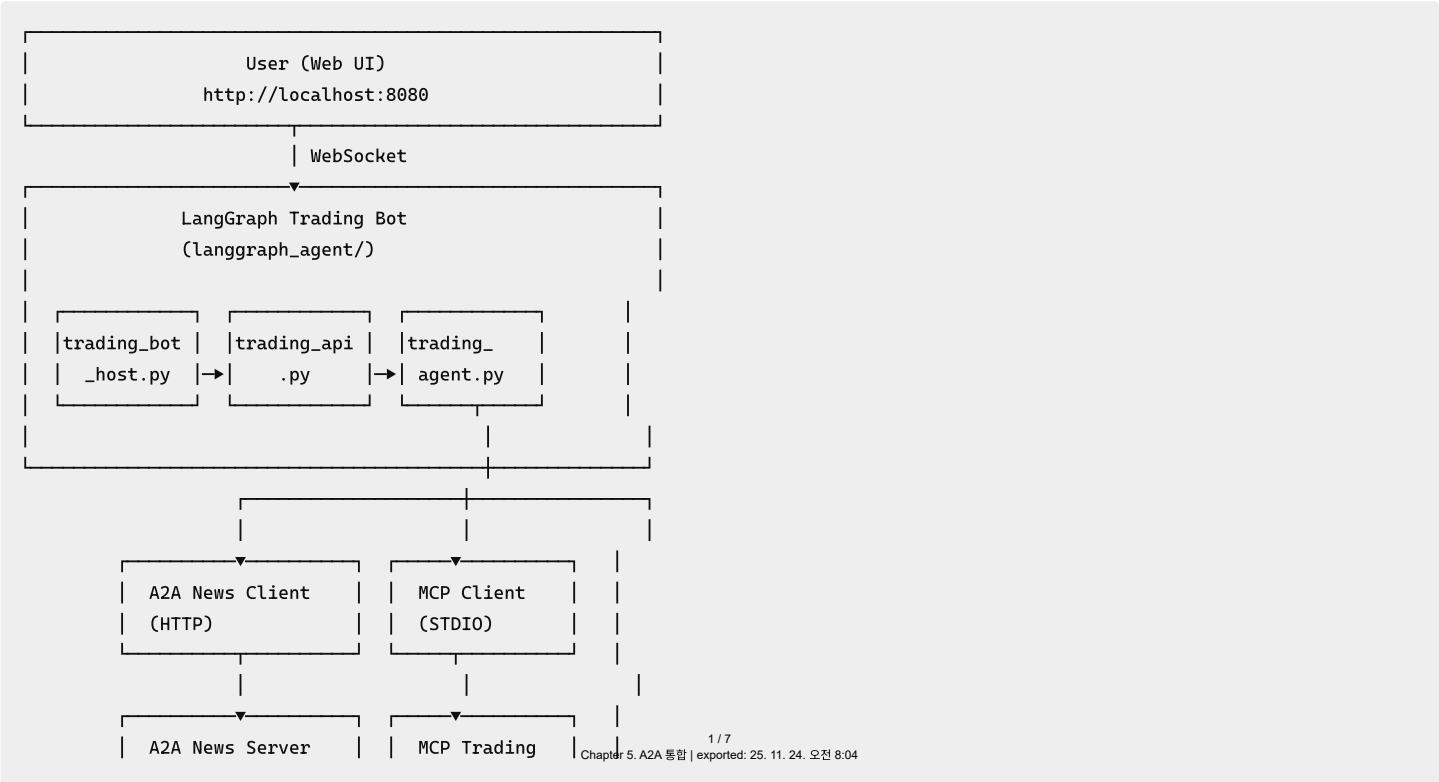


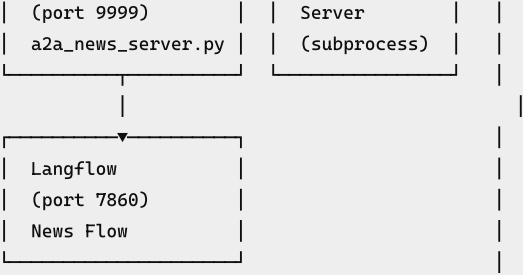
왜 A2A가 필요한가?

- 1. **전문화**: 각 Agent가 자신의 강점 분야에 집중
 - Trading Agent: 거래 로직, 리스크 관리
 - News Agent: 뉴스 수집, 감성 분석
- 2. **재사용성**: 한 번 만든 Agent를 여러 곳에서 활용
 - Langflow News Agent를 Claude, 다른 LangGraph Agent도 사용 가능
- 3. **독립성**: Agent 간 느슨한 결합
 - 각 Agent를 독립적으로 개발, 배포, 업그레이드

🏗️ 시스템 아키텍처

전체 구조





데이터 흐름

```
1. User: "AAPL 분석해줘"
   ↓ HTTP POST
2. Trading API (/api/trade)
   ↓ async call
3. Trading Agent (LangGraph)
   ↳ A2A Client.fetch("AAPL")
   |   ↓ HTTP (A2A Protocol)
   |   A2A Server (port 9999)
   |   ↓ HTTP REST
   |   Langflow (GDELTA + LLM)
   |   ↓ 뉴스 요약 반환
   |   ← "AAPL 신제품 발표, 긍정적 반응"
   |
   ↳ MCP Client.analyze_market_trend("AAPL")
   |   ↓ STDIO (JSON-RPC)
   |   MCP Server
   |   ↓ 시뮬레이션 데이터
   |   ← "상승 추세, 거래량 증가"

4. LLM 종합 판단: "뉴스 긍정 + 시장 상승 → BUY 추천"
   ↓ interrupt()
5. User: 승인/거부 (WebSocket)
   ↓ resume()
6. execute_trade() → 완료
```

🔍 A2A Discovery: Bootstrap 패턴

문제: Agent가 서로를 어떻게 찾나?

Static 방식 (비추천):

```
# 서버 정보를 하드코딩
client = create_client(
    url="http://localhost:9999",
    capabilities=["streaming", "tasks"],
    skills=["news_research"]
)
```

❌ 서버 변경 시 클라이언트도 수정 필요

Dynamic Discovery (A2A 표준):

```
# 1) Bootstrap Card로 시작 (URL만 알면 됨)
bootstrap = AgentCard(url="http://localhost:9999", name="bootstrap")
tmp_client = factory.create(bootstrap)

# 2) 서버에게 정보 요청
real_card = await tmp_client.get_card()
# → 서버가 자신의 capabilities, skills 등 반환

# 3) 실제 통신
client = factory.create(real_card)
response = await client.send_message(...)
```

✅ 서버가 자신의 정보를 제공 → 유연함

우리 구현

```
class NewsA2AClient:
    def _build_bootstrap_card(self) -> AgentCard:
        """최소 정보만 담은 부트스트랩 카드"""
        return AgentCard(
            url=self.base_url,
            preferred_transport=TransportProtocol.jsonrpc,
            supports_authenticated_extended_card=False,
            description="bootstrap card for server discovery",
            version="0.0.0", # 임시
            name="bootstrap", # 임시
        )

    async def fetch(self, ticker: str):
        # 1단계: Bootstrap
        bootstrap_card = self._build_bootstrap_card()
        tmp_client = factory.create(bootstrap_card)

        # 2단계: Discovery
        real_card = await tmp_client.get_card()

        # 3단계: 실제 통신
        client = factory.create(real_card)
        response = await client.send_message(...)
```

A2A 메시지 구조

Request (Client → Server)

```
Message(
  role=Role.user,
  parts=[
    Part(root=DataPart(data={
      "type": "news_research_request",
      "ticker": "AAPL",
      "lang": "ko-KR",
      "country": "KR"
    })))
  ],
  message_id="uuid-1234"
)
```

Response (Server → Client)

```
Message(
  role=Role.agent,
  parts=[
    Part(root=DataPart(data={
      "news": {
        "summary": "AAPL 신제품 발표...",
        "ticker": "AAPL",
        "generated_at": "2025-11-23T..."
      }
    })))
  ],
  parent_message_id="uuid-1234"
)
```

구현 상세

1. A2A Server (Langflow 래퍼)

파일: a2a_news_server.py

```
class NewsResearchExecutor(AgentExecutor):
    """A2A Agent Executor - Langflow를 A2A 프로토콜로 노출"""

    def __init__(self, adapter: LangFlowRESTAdapter):
        self.adapter = adapter # Langflow REST API 호출

    async def execute(self, context: RequestContext,
                     event_queue: EventQueue) -> None:
        # 1. 요청 파싱
        req = self._parse_request(context)
        ticker = req.get("ticker", "UNKNOWN")

        # 2. Langflow 호출
        query = f"{ticker} 뉴스 동향 요약"
        raw_text = await self.adapter.run(query)

        # 3. 결과 변환
        result = self.adapter.to_standard_schema(raw_text)

        # 4. A2A 메시지로 전송
        msg = new_agent_parts_message(
            parts=[Part(root=DataPart(data=result))]
        )
        await event_queue.enqueue_event(msg)
```

핵심 포인트:

- AgentExecutor: A2A 표준 인터페이스
- EventQueue: 비동기 응답 전송
- DataPart: Structured data 전달

2. A2A Client (Trading Bot에서 호출)

파일: a2a_news_client.py

```
class NewsA2AClient:
    async def fetch(self, ticker: str) -> Dict[str, Any]:
        """A2A 서버에서 뉴스 조회"""

        async with httpx.AsyncClient(timeout=self.timeout) as http:
            config = ClientConfig(
                streaming=False,
                httpx_client=http,
                supported_transports=[TransportProtocol.jsonrpc]
            )
            factory = ClientFactory(config=config)

            # Bootstrap → Discovery → Communication
            bootstrap = self._build_bootstrap_card()
            tmp_client = factory.create(bootstrap)
            real_card = await tmp_client.get_card()
            client = factory.create(real_card)

            # 메시지 전송
            req_msg = self._build_request_message(ticker)
            async for event in client.send_message(req_msg, ...):
                news = self._extract_news_from_message(event)
                if news:
                    return {"news": news}

            return {}
```

3. Trading Agent 통합

파일: langgraph_agent/trading_agent.py

```

async def run_trading_analysis(agent, ticker: str, ...):
    # 1. 뉴스 조회 (A2A)
    news_data = await _fetch_news_for_ticker(ticker)
    if news_data and news_data.get("news"):
        news_summary = news_data["news"]["summary"][:500]

    # 2. 프롬프트에 뉴스 포함
    task_instruction = f"""
{ticker} 종목 거래 분석:
{news_summary} ← 여기에 뉴스 요약 추가

다음 단계를 따라주세요:
1. analyze_market_trend 도구로 시장 분석
2. 뉴스 + 시장 데이터 종합 판단
3. BUY/SELL/HOLD 결정
...
"""

    # 3. Agent 실행
    messages = [HumanMessage(content=task_instruction)]
    async for chunk in agent.astream({"messages": messages}, ...):
        # ... 처리

```

실행 방법

1. 환경 설정

```

# .env 파일 작성
cp env.example .env

# 필수 환경변수 설정
OPENAI_API_KEY=sk-...

# Langflow + A2A 설정
LANGFLOW_BASE_URL=http://localhost:7860
LANGFLOW_FLOW_ID=your-flow-id-here
LANGFLOW_API_KEY=your-api-key-here
A2A_SERVER_PORT=9999

```

2. Langflow 실행 (별도 환경)

```

# 옵션 A: WSL에서 (권장)
wsl -e langflow run

# 옵션 B: 별도 venv
python -m venv venv-langflow
source venv-langflow/bin/activate # Windows: venv-langflow\Scripts\activate
pip install langflow
langflow run
deactivate

```

Langflow UI:

1. 브라우저: `http://localhost:7860`
2. News Research Flow 생성/Import
3. API Key 발급 (Settings → API Keys)
4. Flow ID 확인 (URL에서)

3. A2A News Server 실행

```

# 터미널 1: A2A Server
uv run python a2a_news_server.py

# 확인: http://localhost:9999 접속 시 A2A 서버 응답

```

4. Trading Bot 실행

```
# 터미널 2: Trading Bot
uv run python langgraph_agent/trading_bot_host.py

# 브라우저: http://localhost:8080
```

5. 테스트

- 1. 종목 선택: AAPL, MSFT, NVDA 등
- 2. 분석 스타일:
 - Default: 일반 분석
 - Neutral Analyst: 무조건 HOLD 추천 (MCP Prompt)
- 3. 분석 요청 버튼 클릭
- 4. 결과 확인:
 - 뉴스 요약 (A2A → Langflow)
 - 시장 분석 (MCP → trading_mcp_server)
 - 종합 판단 (LLM)
- 5. 승인/거부: BUY/SELL 추천 시 승인 필요

디버깅

A2A Server 로그 확인

```
# a2a_news_server.py에서
log.setLevel(logging.DEBUG)

# 출력 예시 :
# >>> [SERVER] ticker=AAPL
# >>> [SERVER] raw_text={"outputs": [...]}
# >>> [SERVER] result={"news": {...}}
# >>> [SERVER] enqueue 완료
```

A2A Client 로그 확인

```
# trading_agent.py에서
print(f"📰 뉴스 조회 중: {ticker}")
print(f"✅ 뉴스 조회 완료: {len(summary)} 글자")
```

네트워크 확인

```
# A2A Server 포트 확인
curl http://localhost:9999

# Langflow 확인
curl http://localhost:7860/health
```

💡 핵심 개념 정리

A2A vs MCP

특징	A2A	MCP
목적	Agent 간 통신	Tool 제공
프로토콜	HTTP + JSON-RPC	STDIO/SSE/HTTP
사용 사례	Agent 협력	LLM에게 도구 제공
데이터	복잡한 구조화 데이터	함수 호출 + 결과
Discovery	Bootstrap + get_card()	Server capabilities

우리 시스템에서

A2A: LangGraph ↔ Langflow

- 뉴스 분석 결과 주고받기
- Agent 독립성 유지

MCP: LangGraph ↔ Trading Tools

- analyze_market_trend()
- execute_trade()
- LLM이 도구로 사용

실습 과제

1. A2A 흐름 이해

- Bootstrap Card의 역할 확인
- get_card() 응답 구조 파악
- Message Parts 구조 이해

2. 커스터마이징

- 다른 종목 추가 (한국 주식: 005930.KS)
- 뉴스 필터링 (긍정/부정 뉴스만)
- 추가 데이터 포함 (감성 점수 등)

3. 에러 처리

- Langflow 미실행 시 처리
- A2A Server 타임아웃 처리
- 뉴스 없을 때 Fallback

참고 자료

- [A2A Protocol Specification](#)
- [python-a2a Library](#)
- [LangGraph Documentation](#)
- [FastMCP Documentation](#)